

Spring Security

- ❖ Security is very important for every web application.
- ❖ To protect our application & application data we need to implement security logic.
- ❖ Spring Security is one of the module of spring framework
- ❖ Spring Security concept we can use to secure our web applications / REST APIs.
- ❖ To implement security, we need to know about two concepts
 - 1) Authentication
 - 2) Authorization
- ❖ Authentication means verifying who can access our application. (eg. login credentials)
- ❖ Authorization means verifying which user can access which functionality.

Working with Spring Security

- ❖ To secure our spring boot application we need to add below starter in pom.xml file

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

- ❖ Note-1: When we add this dependency in pom.xml file then by default our application will be secured with "http basic authentication".
- ❖ Note-2: It will generate random password to access our application.
- ❖ We need to use below credentials to access our application

Username : user

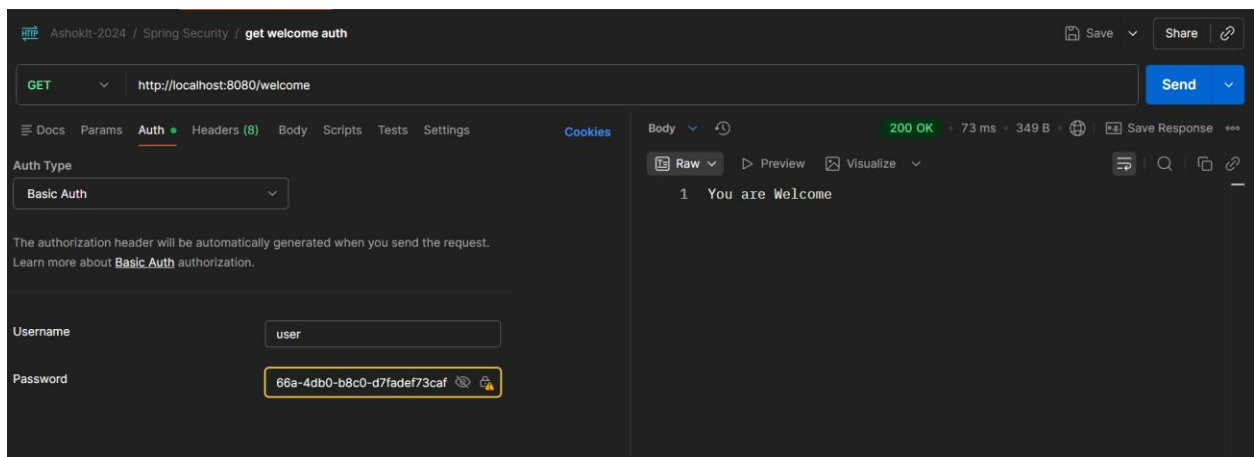
Password : <copy the pwd from console>

- ❖ When we access our application url in browser then it will display "Login Form" to authenticate our request.
- ❖ To access secured REST API from postman, we need to set Auth values in POSTMAN to send the request.

Auth : Basic Auth

Username : user

Password : <copy-from-console>



How to override Spring Security Default Credentials

- ❖ To override Default credentials we can configure security credentials in "application.properties" file or "application.yml" file like below

```
spring.security.user.name=root  
spring.security.user.password=root
```

How to secure specific URL Patterns

- ❖ When we add 'security-starter' in pom.xml then it will apply security filter for all the HTTP methods of our application.
- ❖ But in reality we need to secure only few methods not all methods in our application.

```
/login-page : security not required (anyone can access)
/transfer : security required
/balance : security required
/about-us : security not required
/contact-us : security not required
```

- ❖ In order to achieve above requirement we need to Customize Security Configuration in our project like below.

```
package com.ashokit.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWeb
Security;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class AppSecurityConfigure {

    // SecurityFilterChain
```

```

@Bean
public SecurityFilterChain securityConfig(HttpSecurity httpSecurity)
throws Exception {

    httpSecurity.authorizeHttpRequests(req -> {
        req.requestMatchers("/welcome", "/about-
us", "/contact-us")

        .permitAll()
        .anyRequest()
        .authenticated();
    }).httpBasic(Customizer.withDefaults())
        .formLogin(Customizer.withDefaults());

    return httpSecurity.build();
}
}

```

AshokIt-2024 / Spring Security / get welcome no auth

GET http://localhost:8080/welcome

Auth Type: No Auth

Body: 1 You are Welcome

200 OK · 6 ms · 349 B

This request does not use any authorization.

AshokIt-2024 / Spring Security / get greet basic auth

GET http://localhost:8080/greet

Auth Type: Basic Auth

Username: root

Password: [masked]

Body: 1 Good Morning..!

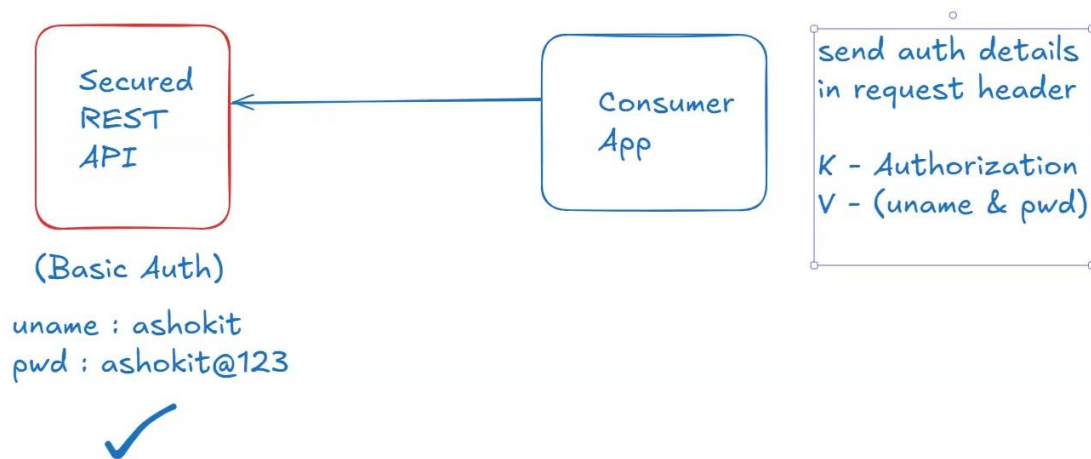
200 OK · 379 ms · 349 B

The authorization header will be automatically generated when you send the request. Learn more about **Basic Auth** authorization.

- ❖ Requirement-1 : Develop REST API with Http Basic Authentication and configure auth credentials in application.properties file or use in-memory authentication.

Note: Test this rest api from browser and from postman.

- ❖ Requirement-2 : Develop Consumer application to access above rest api (secured).



Spring Security In-Memory Authentication

- ❖ In Memory Authentication means storing user credentials in the program for Authentication Purpose.

Note: This is used only for practice purpose, not recommended in real-time.

```

@Configuration
@EnableWebSecurity
public class AppSecurityConfigure {
  
```

```

@Bean
public InMemoryUserDetailsManager inMemoryUserDetailsManager(){
    UserDetails u1= User.withDefaultPasswordEncoder()
        .username("deep")
        .password("deep")
        .build();

    UserDetails u2=User.withDefaultPasswordEncoder()
        .username("raju")
        .password("raju@123")
        .build();

    return new InMemoryUserDetailsManager(u1,u2);
}

```

Rest Template with Basic Authentication Header

```

package com.ashokit.config;

import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder){
        return builder
            .basicAuthentication("deep","deep")
            .build();
    }
}

@Service
public class AppClientService {

    public static final String
    GREET_API_URL="http://localhost:8080/greet";
}

```

```

@Autowired
private RestTemplate restTemplate;

public String callGreetApiFromRestTemplate(){
    return restTemplate.getForObject(GREET_API_URL, String.class);
}
}

```

WebClient with Basic Authentication Header

```

package com.ashokit.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.client.WebClient;

@Configuration
public class WebClientConfig {

    public static final String BASE_URL="http://localhost:8080";

    @Bean
    public WebClient webClient(){

        return WebClient.builder()
            .baseUrl(BASE_URL)
            .defaultHeaders(httpHeaders ->
                httpHeaders.setBasicAuth("deep","deep")
            ).build();

    }

}

@Service
public class AppClientService {

    @Autowired
    private WebClient webClient;
}

```



```

public String callGreetApiFromWebClient(){
    return webClient.get()
        .uri("/greet")
        .retrieve()
        .bodyToMono(String.class)
        .block();
}
}

```

Login and Registration using Spring Security

❖ Develop Spring boot rest api with below 2 functionalities using Spring Security.

1) User Registration (name, email, pwd and phno)

2) User Login (email, pwd)

❖ Note-1: When user register, we need to store user data in database table by encrypting user pwd.

❖ Note-2: When user try to login, if credentials are valid send welcome "msg" as response. If credentials are invalid then send "Invalid Credential" msg as response.

Git Repo :: https://github.com/ashokitschool/springboot_register_login_security.git

Development Process

❖ 1) Create Boot app with required dependencies

web-starter

data-jpa-starter

mysql

security-starter

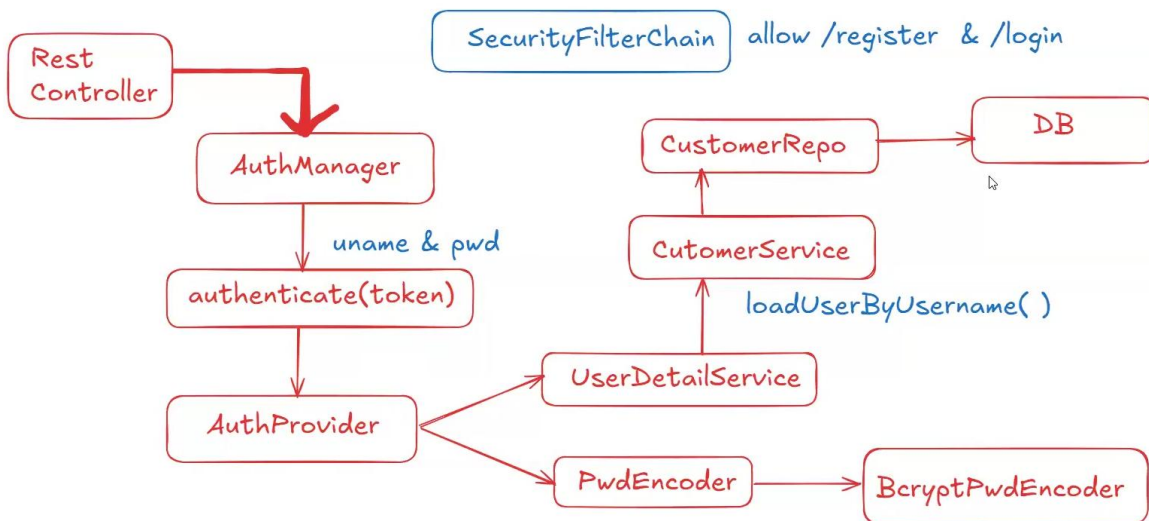
devtools

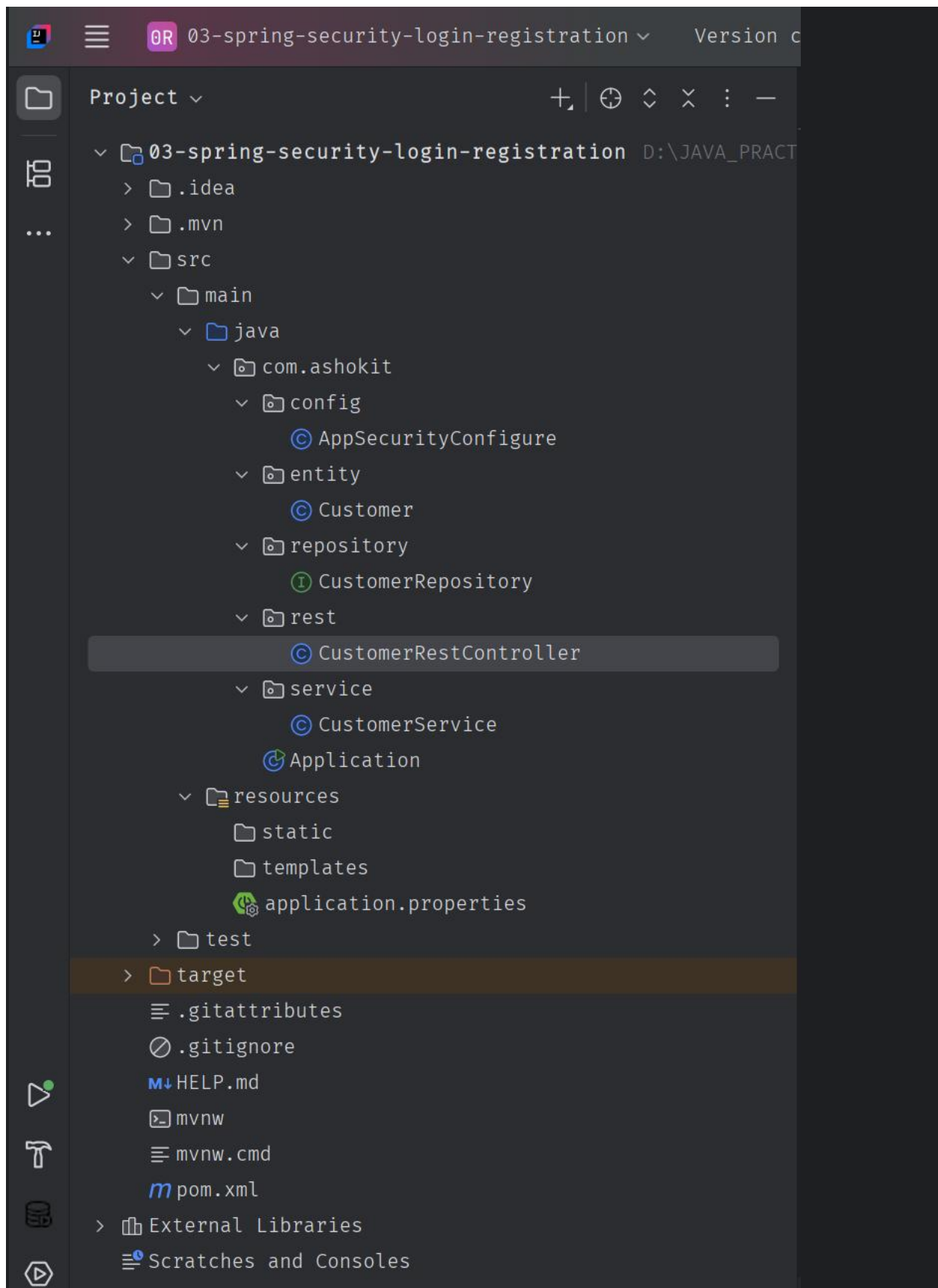
- ❖ 2) Configure Data Source properties in application.properties file
- ❖ 3) Create Entity class & Repository interface
- ❖ 4) Create CustomerService class by implementing UserDetailsService class
- ❖ 5) Create Security Config Class
- ❖ 6) Create RestController with required methods
- ❖ 7) Run the application and test it

```

{
    "name": "Sunil",
    "phno" : 6686868,
    "email" : "sunil@gmail.com",
    "pwd" : "sunil@1233"
}

```





```

package com.ashokit.rest;

import com.ashokit.entity.Customer;
import com.ashokit.repository.CustomerRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CustomerRestController {

    @Autowired
    private CustomerRepository customerRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Autowired
    private AuthenticationManager authenticationManager;

    @PostMapping("/login")
    public ResponseEntity<String> loginCheck(@RequestBody Customer
customer){
        System.out.println("LoginCheck called...");
        UsernamePasswordAuthenticationToken
usernamePasswordAuthenticationToken=
            new
UsernamePasswordAuthenticationToken(customer.getEmail(),customer.getPasswo
rd());

        try {
            Authentication authenticate =
authenticationManager.authenticate(usernamePasswordAuthenticationToken);

            if(authenticate.isAuthenticated())
                return new ResponseEntity<>("Welcome to Ashok IT.. Login
Successfully..",HttpStatus.OK);

```

```

        } catch (Exception exception){
            exception.printStackTrace();
        }

        return new ResponseEntity<>("Invalid Login
Credentials",HttpStatus.BAD_REQUEST);
    }

    @PostMapping("/register")
    public ResponseEntity<Customer> saveCustomer(@RequestBody Customer
customer){

        String encodePassword =
passwordEncoder.encode(customer.getPassword());
        customer.setPassword(encodePassword);

        Customer customerSave = customerRepository.save(customer);

        return new ResponseEntity<>(customerSave, HttpStatus.CREATED);
    }
}
}
}

-----

package com.ashokit.service;

import com.ashokit.entity.Customer;
import com.ashokit.repository.CustomerRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Collections;

@Service
public class CustomerService implements UserDetailsService {

    @Autowired
    private CustomerRepository customerRepo;

    // internally AuthenticationProvider will call this method and load
customer record

```

```

    @Override
    public UserDetails loadUserByUsername(String email) throws
UsernameNotFoundException {
        System.out.println("CustomerService - loadUserByUsername() is
called...");

        Customer user = customerRepo.findByEmail(email);

        return new User(user.getEmail(),user.getPassword(),
Collections.emptyList());
    }
}

```

```

package com.ashokit.repository;

import com.ashokit.entity.Customer;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

```

```

@Repository
public interface CustomerRepository extends
JpaRepository<Customer,Integer> {

    public Customer findByEmail(String email);

}

```

```

package com.ashokit.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

```

```

@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer cid;

    private String name;

    private Long phoneNum;
}

```

```

    private String email;

    private String password;
}

-----

package com.ashokit.config;

import com.ashokit.service.CustomerService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class AppSecurityConfigure {

    @Autowired
    private CustomerService customerService;

    @Bean
    public PasswordEncoder passwordEncoder(){
        return new BCryptPasswordEncoder();
    }

    // load customer db record and perform to authenticate
    @Bean
    public AuthenticationProvider authenticationProvider(){

```

```

        System.out.println("AppSecurityConfigure - AuthenticationProvider
Called...");
        DaoAuthenticationProvider authenticationProvider=new
DaoAuthenticationProvider();

        authenticationProvider.setUserDetailsService(customerService);
        authenticationProvider.setPasswordEncoder(passwordEncoder());

        return authenticationProvider;
    }

    // AuthenticationManager is used to perform authentication and that
internally talk to authenticationProvider
    // Password comparison match by AuthenticationManager
    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration config) throws Exception
{
        System.out.println("AppSecurityConfigure - AuthenticationManager
Called...");
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity
httpSecurity) throws Exception {

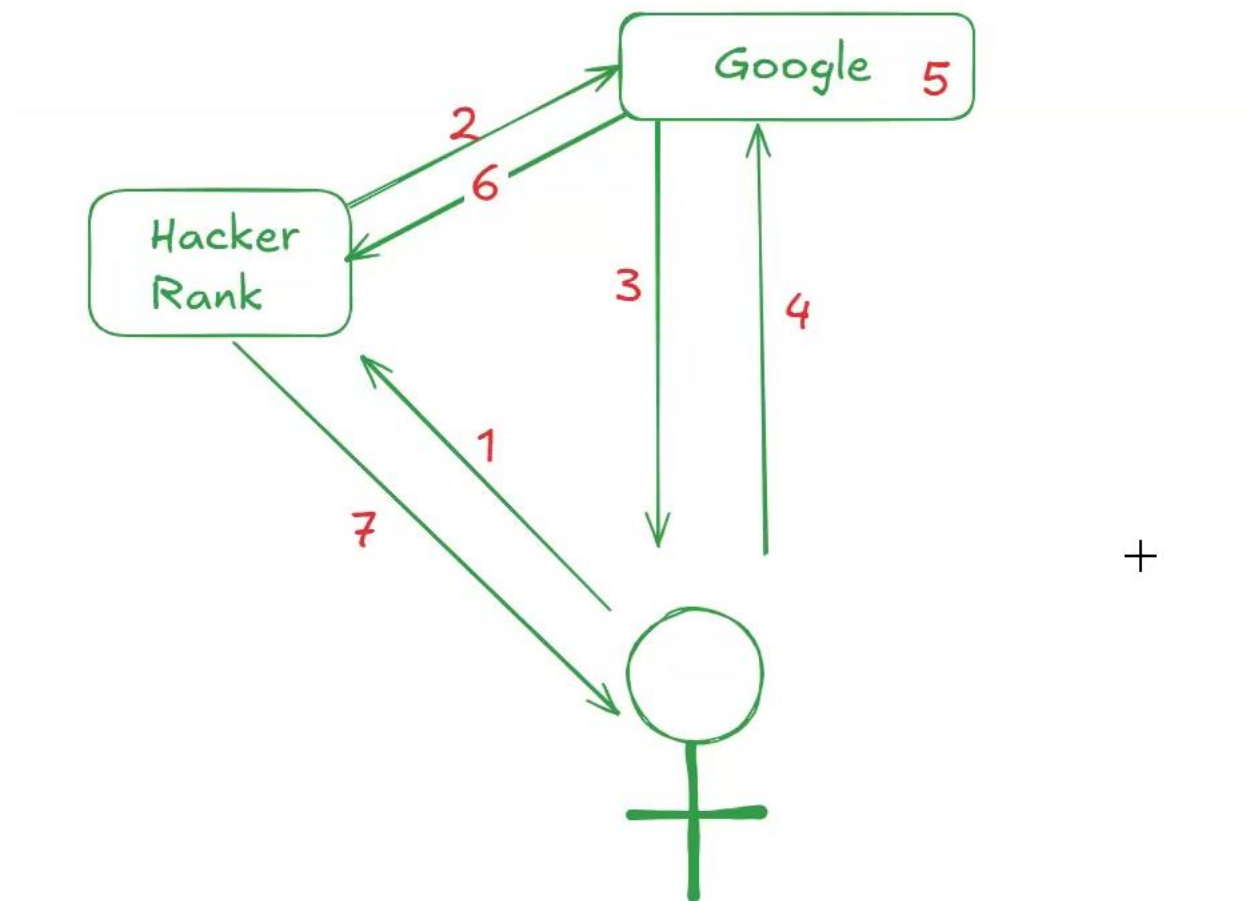
        httpSecurity.authorizeHttpRequests(req -> {
            req.requestMatchers("/register","/login")
                .permitAll()
                .anyRequest()
                .authenticated();
        }).httpBasic(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable());

        return httpSecurity.build();
    }
}

```


Spring Boot with OAuth 2.0

- ❖ OAuth 2.0 is a secure way to let an app access your data without sharing your password.
- ❖ OAuth 2.0 lets apps access user data securely using tokens instead of passwords.



- ❖ 1) Create OAuth app in github.com

(Login --> Profile --> Settings --> Developer Settings --> OAuth Apps --> Create App --> Copy Client ID & Client Secret)

Client ID : Ov23liNCO4r8dB0PIh3n

Client Secret : bea8de3490ef47af306573d315dbaf7b3fd0de9b

- ❖ 2) Create Spring Boot application with below dependencies
 - a) web-starter
 - b) security-starter
 - c) oauth-client

- ❖ 3) Create Rest Controller with method

```
@RestController
public class MsgRestController {

    @GetMapping("/")
    public String welcome(){
        return "Welcome to Ashok IT";
    }

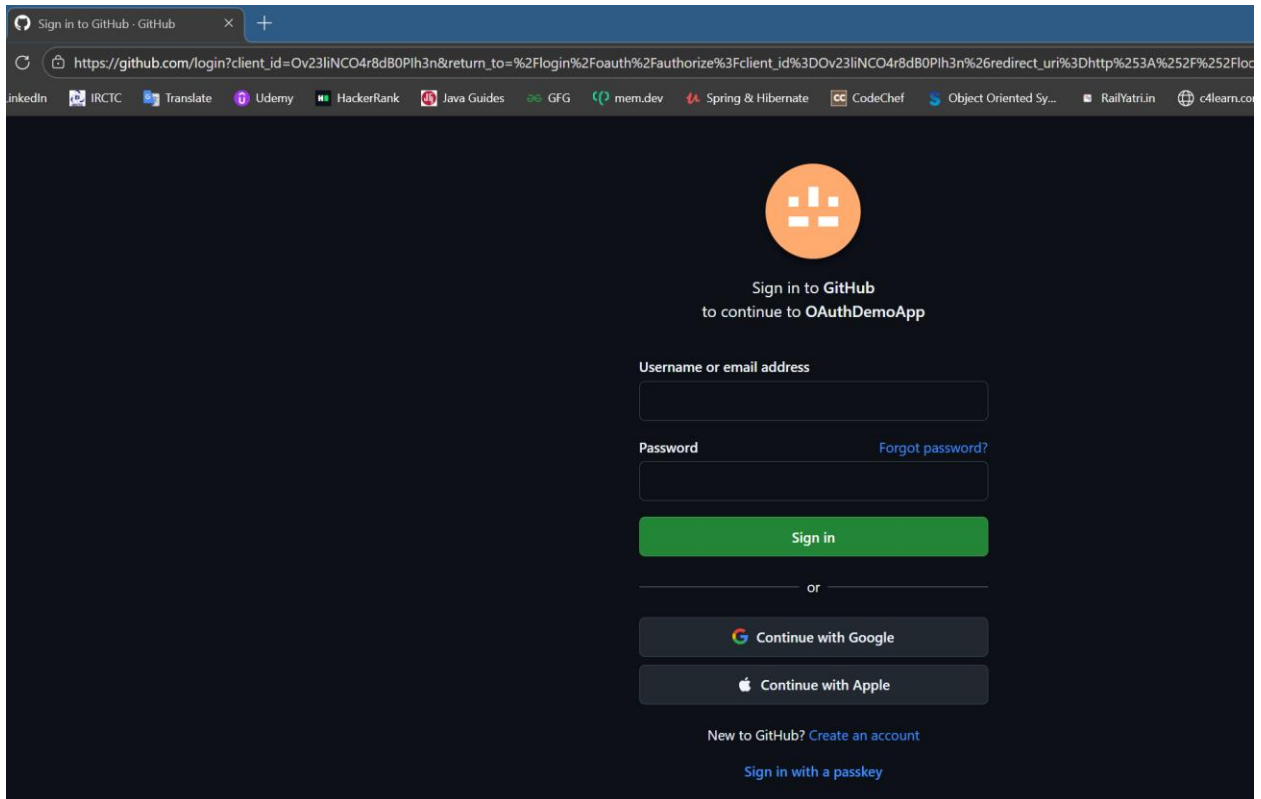
}
```

- ❖ 4) Configure GitHub OAuth App client id & client secret in application.yml file like below

```
spring:
  application:
    name: 04-spring-security-oauth-github
  security:
    oauth2:
      client:
        registration:
          github:
            client-id: 0v23liNC04r8dB0PIh3n
            client-secret: bea8de3490ef47af306573d315dbaf7b3fd0de9b
```

- ❖ 5) Run the application and test it

```
{
  "name": "Sunil",
  "phno" : 6686868,
  "email" : "sunil@gmail.com",
  "pwd" : "sunil@1233"
}
```



Assignment : Spring Boot with oAuth using google account. Get username also from google and display that in response.

Spring Boot with JWT

- ❖ JWT stands for JSON Web Tokens.
- ❖ WT official Website : <https://jwt.io/>
- ❖ Below is the sample JWT Token

```
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQsw5c
```

- ❖ JWT contains below 3 parts

- 1) Header
- 2) Payload
- 3) Signature

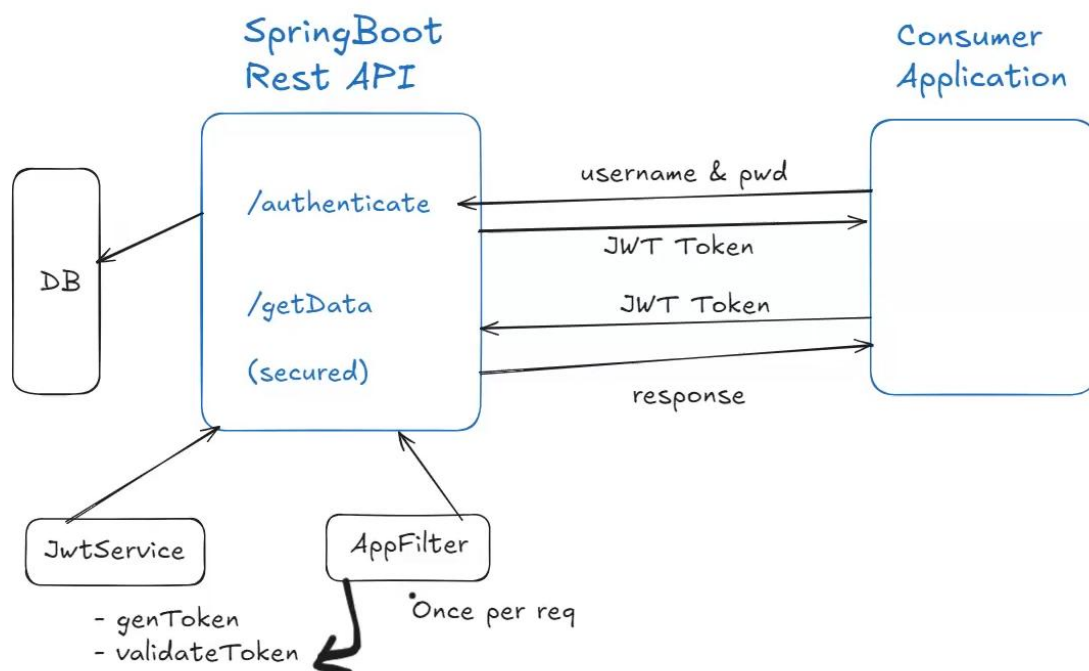
Note: JWT 3 parts will be separated by using dot(.)

Note: Client application should send JWT Token to provider in below format

Authorization=Bearer

eyJhbGciOiJIUzI1NiJ9.eyJzdWwiOiJhbmlsQGdtYWlsLmNvbSIsImhhdCI6MTcyOTA1NzcxNiwiZXhwIjoxNzI5MDYxMzE2fQ.SJ1yonGgN85h7MewbsygPS8pN2JSHRn-6ICJ7bJVbvQ

❖ Git Hub Repo : https://github.com/ashokitschool/SpringBoot_JWT_App.git



1) JWT Token generation (JwtService.java)

- generateToken(String uname)
- validateToken(String uname)

2) JWT Token validation Filter (AppFilter.java) - OncePerRequest

- check Authorization header presence
- retrieve bearer token from header
- validate token
- if token is valid, update security context to process req

3) Customize SecurityFilterChain

- permit /api/register & /api/login urls
- authenticate any other request

Microservices with JWT Security

Git Hub Repo : https://github.com/ashokitschool/Microservices_Security.git

❖ Auth-Service contains functionality for user registration and user login with MySQL DB.

- When user register then user record will be inserted into DB table.

- When user login successfully then it will generate JWT token and send to user as response.

❖ API-Gateway contains logic to validate the token using Filter.

Note: In API-Gateway we have added routings for our microservices along with Filter.

/welcome => AuthFilter => Welcome-API

/greet => AuthFilter => Greet-API

/auth/** => Auth-API

Summary

- 1) What is Spring Security
- 2) Authentication & Authorization
- 3) Basic Authentication
- 4) In Memory authentication
- 5) Securing Specific URLs with Customization
- 6) Login & Registration with Spring Security
- 7) Spring Boot with JWT Integration
- 8) Microservices Security with JWT
- 9) SpringBoot with OAuth 2.0 (Git Hub Login)